Problem 1

Part one

```
import pandas as pd

# File path to the image
file_path = '/Users/yaseen/Library/CloudStorage/OneDrive-University%20Ramon%20Llull/Systems Optimization/Final Exam/Final%20Exam%20Q4_Data.xlsx'

# Load the Excel file into a DataFrame
df = pd.read_excel(file_path)

# Convert columns to lists
items = df['Item'].tolist()
weights = df['Weight (pounds)'].tolist()
values = df['Value'].tolist()

# Displaying the lists
print("Items:", items)
print("Weights:", weights)
print("Values:", values)

Items: ['Item 1', 'Item 2', 'Item 3', 'Item 4', 'Item 5', 'Item 6', 'Item 7', 'Item 8', 'Item 9', 'Item 10', 'Item 11', 'Item 12', 'Item 13', 'Item 14', 'Item 15', 'Item 16', 'Item 17', 'Item 18', 'Item 19', 'Item 20']
Weights: [4, 4, 5, 6, 6, 7, 7, 8, 9, 9, 9, 10, 10, 11, 11, 12, 12, 13, 14, 15]
Values: [5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24]

from gurobipy import *
import pandas as pd

# File path to the image
file_path = '/Users/yaseen/Library/CloudStorage/OneDrive-University%20Ramon%20Llull/Systems Optimization/Final Exam/Final%20Exam%20Q4_Data.xlsx'

# Load the Excel file into a DataFrame
df = pd.read_excel(file_path)

# Convert columns to lists
items = df['Item'].tolist()
weights = df['Weight (pounds)'].tolist()
values = df['Value'].tolist()

# Displaying the lists
print("Items:", items)
print("Weights:", weights)
print("Values:", values)

m = Model("knapsack")

m.setParam('OutputFlag', 0)

x = m.addVars(items, vtype=GRB.BINARY, name="x")

allowed_weight = 15

# Add weight constraint
for i in items:
    quicksum(x[item] * weights[i] for i, item in enumerate(items)) <= allowed_weight,
    name="weight_constraint"

# Set the objective to maximize total profit
m.setObjective(quicksum(x[item] * values[i] for i, item in enumerate(items)), GRB.MAXIMIZE)

m.optimize()

# Print the solution
if m.status == GRB.OPTIMAL:
    print("Optimal solution found:")
    for i, item in enumerate(items):
        if x[item].X > 0:  # If the decision variable value is close to 1, the item is selected
            print(f"{item}: Included (Weight = {weights[i]}, Value = {values[i]})")
else:
    print("Not optimal solution found.")

Optimal solution found:
Item 1: Included (Weight = 4, Value = 5)
Item 2: Included (Weight = 4, Value = 6)
Item 3: Included (Weight = 5, Value = 7)
Item 4: Included (Weight = 6, Value = 8)
Item 5: Included (Weight = 6, Value = 9)
Item 6: Included (Weight = 7, Value = 10)
Item 7: Included (Weight = 7, Value = 11)
Item 8: Included (Weight = 8, Value = 12)
Item 9: Included (Weight = 9, Value = 13)
Total Value: 34.0
```

Part Two

```
# Create a Gurobi model
m = Model("knapsack_two_bags")

m.setParam('OutputFlag', 0)

# Define weight capacities for the two bags
weight_bag1 = 15
weight_bag2 = 10

# Add binary decision variables for each item in both bags
x_bag1 = m.addVars(items, vtype=GRB.BINARY, name="x_bag1")
x_bag2 = m.addVars(items, vtype=GRB.BINARY, name="x_bag2")

# Add weight constraints for each bag
m.addConstr(
    quicksum(x_bag1[item] * weights[i] for i, item in enumerate(items)) <= weight_bag1,
    name="weight_constraint_bag1"
)
m.addConstr(
    quicksum(x_bag2[item] * weights[i] for i, item in enumerate(items)) <= weight_bag2,
    name="weight_constraint_bag2"
)

# Ensure that each item is assigned to at most one bag
m.addConstr(
    quicksum(x_bag1[item] + x_bag2[item] <= 1 for item in items),
    name="assignment_constraint"
)

# Set the objective to maximize the total value of items in both bags
m.setObjective(
    quicksum(x_bag1[i] + x_bag2[item] for i, item in enumerate(items)),
    GRB.MAXIMIZE
)

# Optimize the model
m.optimize()

# Print the solution
if m.status == GRB.OPTIMAL:
    print("Optimal solution found:")
    total_weight_bag1 = 0
    total_weight_bag2 = 0
    total_value = 0.0
    for i, item in enumerate(items):
        if x_bag1[item].X > 0:  # If the item is selected for Bag 1
            print(f"{item}: Included in Bag 1 (Weight = {weights[i]}, Value = {values[i]})")
            total_weight_bag1 += weights[i]  # Add the weight to Bag 1 total weight
            total_value += values[i]  # Add the value to the total value
        elif x_bag2[item].X > 0:  # If the item is selected for Bag 2
            print(f"{item}: Included in Bag 2 (Weight = {weights[i]}, Value = {values[i]})")
            total_weight_bag2 += weights[i]  # Add the weight to Bag 2 total weight
    print(f"Total weight in Bag 1: {total_weight_bag1}")
    print(f"Total weight in Bag 2: {total_weight_bag2}")
    print(f"Total value: {total_value}")
else:
    print("No optimal solution found.")

Optimal solution found:
Item 1: Included in Bag 1 (Weight = 4, Value = 5)
Item 2: Included in Bag 1 (Weight = 4, Value = 6)
Item 3: Included in Bag 1 (Weight = 5, Value = 7)
Item 4: Included in Bag 1 (Weight = 6, Value = 8)
Item 5: Included in Bag 1 (Weight = 6, Value = 9)
Item 11: Included in Bag 1 (Weight = 11, Value = 16)
Item 12: Included in Bag 1 (Weight = 12, Value = 17)
Item 13: Included in Bag 1 (Weight = 13, Value = 18)
Item 14: Included in Bag 1 (Weight = 14, Value = 19)
Item 15: Included in Bag 1 (Weight = 15, Value = 20)
Item 16: Included in Bag 2 (Weight = 10, Value = 21)
Item 17: Included in Bag 2 (Weight = 11, Value = 22)
Item 18: Included in Bag 2 (Weight = 12, Value = 23)
Item 19: Included in Bag 2 (Weight = 13, Value = 24)
Total weight in Bag 1: 72
Total weight in Bag 2: 58
Total Value: 90.0
```

Problem 2

Part 1

```
from gurobipy import Model, GRB, quicksum

# Define the data
cities = [1, 2, 3, 4, 5, 6, 7]  # City labels
n = len(cities)

# Travel costs and times for bus (matrix form)
bus_costs = [
    [0, 25, 30, 35, 40, 45, 45],
    [25, 0, 20, 25, 30, 35, 35],
    [30, 20, 0, 15, 20, 25, 25],
    [35, 25, 15, 0, 15, 20, 20],
    [40, 30, 20, 15, 0, 15, 15],
    [45, 35, 25, 20, 15, 0, 10],
    [45, 35, 25, 20, 15, 10, 0]
]
bus_times = [
    [0, 5, 6, 7, 8, 9, 9],
    [5, 0, 4, 5, 6, 7, 7],
    [6, 4, 0, 3, 4, 5, 5],
    [7, 5, 3, 0, 3, 4, 4],
    [8, 6, 4, 3, 0, 3, 3],
    [9, 7, 5, 4, 3, 0, 2],
    [9, 7, 5, 4, 3, 2, 0]
]

# Create the Gurobi model
model = Model("TSP_plane_only")

model.setParam('OutputFlag', 0)

# Decision variables
x = model.addVars(n, n, vtype=GRB.BINARY, name="x_plane")  # Binary for plane routes
u = model.addVars(n, vtype=GRB.CONTINUOUS, lb=1, ub=n, name="u")  # Position variables for subtour elimination

# Objective: Minimize total travel time
model.setObjective(
    quicksum(x[i,j,0] * bus_times[i-1][j-1] for i in range(n) for j in range(n) if i != j),
    GRB.MINIMIZE
)

# Constraints

# 1. Each city must be visited exactly once
for i in range(n):
    model.addConstr(
        quicksum(x[i,j,0] for j in range(n) if j != i) == 1,
        name="visit_once_out_c1"
    )
    model.addConstr(
        quicksum(x[j,i,0] for j in range(n) if j != i) == 1,
        name="visit_once_in_c1"
    )

# 2. Subtour elimination (MTZ constraints)
for i in range(n):
    for j in range(1, n):
        if i != j:
            model.addConstr(
                u[i] - u[j] + n * x[i,j,0] >= n - 1,
                name="subtour_elimination_c2_c3"
            )

# 3. Fix City 1 (hometown) as the starting and ending point
hometown = 0
model.addConstr(quicksum(x[hometown,j,0] for j in range(n) if j != hometown) == 1,
    name="start_at_hometown"
)

model.addConstr(
    quicksum(x[j,hometown,0] for j in range(n) if j != hometown) == 1,
    name="end_at_hometown"
)


# Optimize the model
model.optimize()

# Print the solution
if model.status == GRB.OPTIMAL:
    total_cost = 0.0
    plane_cost[][] = n_plane[n, n] # Initialize plane cost matrix
    for i in range(n):
        print("\nCheapest solution found:")
        print("Visit order: 1 -> {}".format(i+1 if i!=0 else 1))
        print("Total cost (USD): {total_cost:.2f}")
        print("Routes:")
        for j in range(n):
            if x[i,j,0].X > 0:
                print("From {} to {}".format(i+1, j+1))
else:
    print("No optimal solution found.")

Optimal solution found:
Total travel cost: 105.00
Total cost (USD): 105.00
Routes:
From 1 to 4
From 2 to 1
From 3 to 2
From 4 to 3
From 5 to 6
From 6 to 7
From 7 to 5
```

Part 2

```
from gurobipy import Model, GRB, quicksum

# Define the data
cities = [1, 2, 3, 4, 5, 6, 7]  # City labels
n = len(cities)

# Travel costs and times for bus (matrix form)
bus_costs = [
    [0, 25, 30, 35, 40, 45, 45],
    [25, 0, 20, 25, 30, 35, 35],
    [30, 20, 0, 15, 20, 25, 25],
    [35, 25, 15, 0, 15, 20, 20],
    [40, 30, 20, 15, 0, 15, 15],
    [45, 35, 25, 20, 15, 0, 10],
    [45, 35, 25, 20, 15, 10, 0]
]
bus_times = [
    [0, 5, 6, 7, 8, 9, 9],
    [5, 0, 4, 5, 6, 7, 7],
    [6, 4, 0, 3, 4, 5, 5],
    [7, 5, 3, 0, 3, 4, 4],
    [8, 6, 4, 3, 0, 3, 3],
    [9, 7, 5, 4, 3, 0, 2],
    [9, 7, 5, 4, 3, 2, 0]
]

# Create the Gurobi model
model = Model("TSP_plane_only")

model.setParam('OutputFlag', 0)

# Decision variables
x = model.addVars(n, n, vtype=GRB.BINARY, name="x_plane")  # Binary for bus routes
u = model.addVars(n, vtype=GRB.CONTINUOUS, lb=1, ub=n, name="u")  # Position variables for subtour elimination

# Objective: Minimize total travel time
model.setObjective(
    quicksum(x[i,j,0] * bus_costs[i-1][j-1] for i in range(n) for j in range(n) if i != j),
    GRB.MINIMIZE
)

# Constraints

# 1. Each city must be visited exactly once
for i in range(n):
    model.addConstr(
        quicksum(x[i,j,0] for j in range(n) if j != i) == 1,
        name="visit_once_out_c1"
    )
    model.addConstr(
        quicksum(x[j,i,0] for j in range(n) if j != i) == 1,
        name="visit_once_in_c1"
    )

# 2. Subtour elimination (MTZ constraints)
for i in range(n):
    for j in range(1, n):
        if i != j:
            model.addConstr(
                u[i] - u[j] + n * x[i,j,0] >= n - 1,
                name="subtour_elimination_c2_c3"
            )

# 3. Fix City 1 (hometown) as the starting and ending point
hometown = 0
model.addConstr(quicksum(x[hometown,j,0] for j in range(n) if j != hometown) == 1,
    name="start_at_hometown"
)

model.addConstr(
    quicksum(x[j,hometown,0] for j in range(n) if j != hometown) == 1,
    name="end_at_hometown"
)


# Optimize the model
model.optimize()

# Print the solution
if model.status == GRB.OPTIMAL:
    total_time = 0.0
    bus_time[][] = n_bus[n, n] # Initialize bus time matrix
    for i in range(n):
        print("\nCheapest solution found:")
        print("Visit order: 1 -> {}".format(i+1 if i!=0 else 1))
        print("Total travel time: {total_time:.2f} hours")
        print("Routes:")
        for j in range(n):
            if x[i,j,0].X > 0:
                print("From {} to {}".format(i+1, j+1))
else:
    print("No optimal solution found.")

Optimal solution found:
Total travel time: 57.00
Total travel time: 57.00 hours
Routes:
From 1 to 4
From 2 to 1
From 3 to 2
From 4 to 3
From 5 to 6
From 6 to 7
From 7 to 5
```

Part 3

```
from gurobipy import Model, GRB, quicksum

# Define the data
cities = [1, 2, 3, 4, 5, 6, 7]  # City labels
n = len(cities)

# Travel costs and times for bus (matrix form)
bus_costs = [
    [0, 25, 30, 35, 40, 45, 45],
    [25, 0, 20, 25, 30, 35, 35],
    [30, 20, 0, 15, 20, 25, 25],
    [35, 25, 15, 0, 15, 20, 20],
    [40, 30, 20, 15, 0, 15, 15],
    [45, 35, 25, 20, 15, 0, 10],
    [45, 35, 25, 20, 15, 10, 0]
]
bus_times = [
    [0, 5, 6, 7, 8, 9, 9],
    [5, 0, 4, 5, 6, 7, 7],
    [6, 4, 0, 3, 4, 5, 5],
    [7, 5, 3, 0, 3, 4, 4],
    [8, 6, 4, 3, 0, 3, 3],
    [9, 7, 5, 4, 3, 0, 2],
    [9, 7, 5, 4, 3, 2, 0]
]

# Travel costs and times for airplane (matrix form)
plane_costs = [
    [0, 150, 180, 200, 220, 250, 250],
    [150, 0, 120, 140, 160, 180, 180],
    [180, 120, 0, 100, 120, 140, 140],
    [200, 140, 100, 0, 100, 120, 120],
    [220, 160, 120, 100, 0, 100, 100],
    [250, 180, 140, 120, 100, 0, 80],
    [250, 180, 140, 120, 100, 80, 0]
]
plane_times = [
    [0, 2, 2.5, 3, 3.5, 4, 4.5],
    [2, 0, 1.5, 2, 2.5, 3, 3.5],
    [2.5, 1.5, 0, 1.5, 2, 2.5, 3],
    [3, 2, 1.5, 0, 1.5, 2, 2.5],
    [3.5, 2.5, 2, 1.5, 0, 1.5, 2],
    [4, 3, 2.5, 2, 1.5, 0, 1.5],
    [4.5, 3.5, 3, 2.5, 2, 1.5, 0]
]

# Create the Gurobi model
model = Model("TSP_both_modes")

model.setParam('OutputFlag', 0)

# Decision variables
x = model.addVars(n, n, vtype=GRB.BINARY, name="x_bus")  # Binary for bus routes
y = model.addVars(n, n, vtype=GRB.BINARY, name="y")  # Binary for plane routes
u = model.addVars(n, vtype=GRB.CONTINUOUS, lb=1, ub=n, name="u")  # Position variables for subtour elimination

# Objective: Minimize total travel time
model.setObjective(
    quicksum(x[i,j,0] * bus_times[i-1][j-1] for i in range(n) for j in range(n) if i != j),
    GRB.MINIMIZE
)

# Constraints

# 1. Each city must be visited exactly once
for i in range(n):
    model.addConstr(
        quicksum(x_bus[i,j] + y[i,j] for j in range(n) if j != i) == 1,
        name="visit_once_out_c1"
    )
    model.addConstr(
        quicksum(x_bus[j,i] + y[j,i] for j in range(n) if j != i) == 1,
        name="visit_once_in_c1"
    )

# 2. Subtour elimination (MTZ constraints)
for i in range(n):
    for j in range(1, n):
        if i != j:
            model.addConstr(
                u[i] - u[j] + n * (x_bus[i,j] + y[i,j]) >= n - 1,
                name="subtour_elimination_c2_c3"
            )

# 3. Only one mode of transportation per route
for i in range(n):
    for j in range(n):
        if i != j:
            model.addConstr(x_bus[i,j] + y[i,j] <= 1, name="one_mode_c11_c12")

# 4. Total travel time constraint
model.addConstr(
    quicksum(x_bus[i,j,0] * bus_times[i-1][j-1] for i in range(n) for j in range(n) if i != j) +
    quicksum(y[i,j] * plane_times[i-1][j-1] for i in range(n) for j in range(n) if i != j) <= t_max,
    name="time_constraint"
)

# 5. Fix City 1 (hometown) as the starting and ending point
# Ensure that the tour is indeed an 8-city tour
hometown = 0
model.addConstr(
    quicksum(x_bus[hometown,j] + y[hometown,j] for j in range(n) if j != hometown) == 1,
    name="start_at_hometown"
)

model.addConstr(
    quicksum(x_bus[i,hometown] + y[i,hometown] for i in range(n) if i != hometown) == 1,
    name="end_at_hometown"
)


# Optimize the model
model.optimize()

# Print the solution
if model.status == GRB.OPTIMAL:
    total_time = 0.0
    bus_time[][] = n_bus[n, n] # Initialize bus time matrix
    plane_time[][] = n_plane[n, n] # Initialize plane time matrix
    for i in range(n):
        print("\nOptimal solution found:")
        print("Total travel cost: {model.objVal:.2f} USD")
        print("Total travel time: {total_time:.2f} hours")
        print("Routes:")
        for j in range(n):
            if x[i,j,0].X > 0.5:
                print("From {} to {} - Bus".format(i+1, j+1))
            elif y[i,j].X > 0.5:
                print("From {} to {} - Plane".format(i+1, j+1))
else:
    print("No optimal solution found.")

Optimal solution found:
Total travel cost: 840.00 USD
Total travel time: 19.00 hours
Routes:
From 1 to 4
From 2 to 1
From 3 to 2
From 4 to 3
From 5 to 6
From 6 to 7
From 7 to 5
```

Problem 3

```
from gurobipy import Model, GRB, quicksum

# Data
plants = ["P1", "P2"]
cities = ["C1", "C2", "C3"]

cost = [
    ["P1", "C1", 16, [16], "P1", "C1", "P1"],
    ["P1", "C2", 13, [13], "P1", "C2", "P1"],
    ["P1", "C3", 21, [21], "P1", "C3", "P1"],
    ]

prt_capacity = {
    ("P1", "C1"): 11, ("P1", "C2"): 16, ("P1", "C3"): 20,
    ("P2", "C1"): 12, ("P2", "C2"): 15, ("P2", "C3"): 18,
    }

demand = {"C1": 25, "C2": 15, "C3": 40}

# Model
model = Model("ProductionCapacityAndDistribution")

model.setParam('OutputFlag', 0)

# Decision variables
x = model.addVars(plants, cities, vtype=GRB.CONTINUOUS, name="x")  # Amount transported
y = model.addVars(plants, vtype=GRB.CONTINUOUS, name="y")  # Plant production capacity

# Objective: Minimize total cost
model.setObjective(
    quicksum(x[p,c] * cost[i][3][0] for i, p in plants for j in cities),
    GRB.MINIMIZE
)

# Constraints

# 1. Demand constraints
for c in cities:
    model.addConstr(
        quicksum(x[i,c] for i in plants) >= demand[c],
        name="Demand_c1"
    )

# 2. Production capacity constraints
for p in plants:
    model.addConstr(
        quicksum(x[i,j] for j in cities) <= y[i],
        name="Production_capacity_c2"
    )

# 3. Arc capacity constraints
for i, p in plants:
    for j in cities:
        model.addConstr(
            x[i,j] <= prt_capacity[(i, j)],
            name="Port_capacity_c3_c4"
        )


# Optimize
model.optimize()

# Print solution
if model.status == GRB.OPTIMAL:
    print("\nOptimal solution found:")
    print("Total cost: {model.objVal:.2f}")
    for p in plants:
        print("{p} production capacity: {y[p].x:.2f} tons")
    for i, p in plants:
        for j in cities:
            if x[i,j].X > 0:
                print("Transport {x[i,j].x:.2f} tons from {i} to {j}")
else:
    print("No optimal solution found.")

Optimal solution found:
Total cost: 1.00
P1 production capacity: 31.00 tons
P2 production capacity: 0.00 tons
Transport 5.00 tons from P1 to C1
Transport 16.00 tons from P1 to C2
Transport 10.00 tons from P1 to C3
Transport 15.00 tons from P2 to C2
Transport 18.00 tons from P2 to C3
```